The Crown Jewel Finding

Unauthenticated WebLogic wls-wsat RCE
on the Internet-Banking Edge

CRITICAL — CVSS 10.0

Coordinated Vulnerability Disclosure – Priority Finding Summary

This finding documents an unauthenticated, root-yielding remote code execution vulnerability on both public internet-banking edge nodes, chained end-to-end in a controlled lab replication into credential-store decryption, live session hijacking, durable persistence, and internal network pivoting.

Target	Value
Primary hosts	ibank.hbl.com (45.60.79.176) www.ibank.hbl.com (45.60.76.176)
Vulnerable endpoint	/wls-wsat/CoordinatorPortType
Vulnerability class	Unauthenticated Java deserialization (XMLDecoder) RCE
CVE family	CVE-2017-10271 / WS-AT WorkContext deserialization family
Related sibling issues	_async/AsyncResponseService RCE (CVE-2019-2725 family); UDDI Explorer SSRF (CVE-2014-4210)
Preconditions	None — network reachability only
Impact	Root-level code execution on the internet-banking application tier

Prepared for coordinated disclosure. Distribution restricted to the recipient security team and named stakeholders. Do not forward outside the agreed disclosure channel.

Why This Finding Matters

Both internet-banking edge nodes — ibank.hbl.com (45.60.79.176) and www.ibank.hbl.com (45.60.76.176) — respond with HTTP 200 on /wls-wsat/CoordinatorPortType through Imperva, exposing a WS-AT coordination service. The SOAP PortType markers match, byte-for-byte, a build exploited end-to-end under controlled conditions in our lab. The same sweep confirmed two sibling surfaces on the same nodes: a second unauthenticated deserialization RCE in the _async/AsyncResponseService path (CVE-2019-2725 family), and a classic SSRF oracle in the UDDI Explorer (CVE-2014-4210).

Every other serious issue in this engagement requires something first — valid credentials for the VLink portal or Merchant Portal, an unpatched SharePoint farm plus a WAF-bypass flow, or successful password guessing against the Dev IdP. The wls-wsat exposure needs nothing but network reachability, and reachability is already proven. It is the only finding where attacker capability moves directly from “internet outsider” to “arbitrary code on the banking tier” without acquiring credentials, bypassing MFA, or tricking a user — that zero-precondition jump is what makes it the crown jewel.

Why It Executes Code At All

WebLogic's WS-AT coordination service accepts SOAP envelopes whose headers contain a <work:WorkContext> element. Internally, WebLogic hands the contents of that header to java.beans.XMLDecoder — a JDK class whose purpose is to turn XML into live Java objects, including constructor calls and method invocations. This is object instantiation and execution, not parsing or validation.

A crafted WorkContext header causes the server to literally evaluate a process-spawning constructor call, running as the WebLogic process user. Authentication is never consulted because WS-AT is an old, machine-to-machine transaction coordination feature that predates modern exposure hardening.

```
new ProcessBuilder("/bin/sh", "-c", "<attacker command>").start()
```

Oracle's fixes for this family were essentially “stop handing untrusted XML to XMLDecoder” plus serialization filters. On unpatched builds there is no external signal distinguishing “safe” from “vulnerable” — the only way to tell from the outside is to fire a payload and watch for side effects. This was validated against an identical software stack: one POST, root shell, blind execution (HTTP 500 regardless of success), confirmed via out-of-band artifacts. This is a replicated exploit path, not a theoretical one.

Attack Chain — How It Happens End-to-End

The diagram below traces the path from an unauthenticated internet attacker to full compromise of the internet-banking tier, and shows how the sibling exposures found in the same sweep compound the risk.

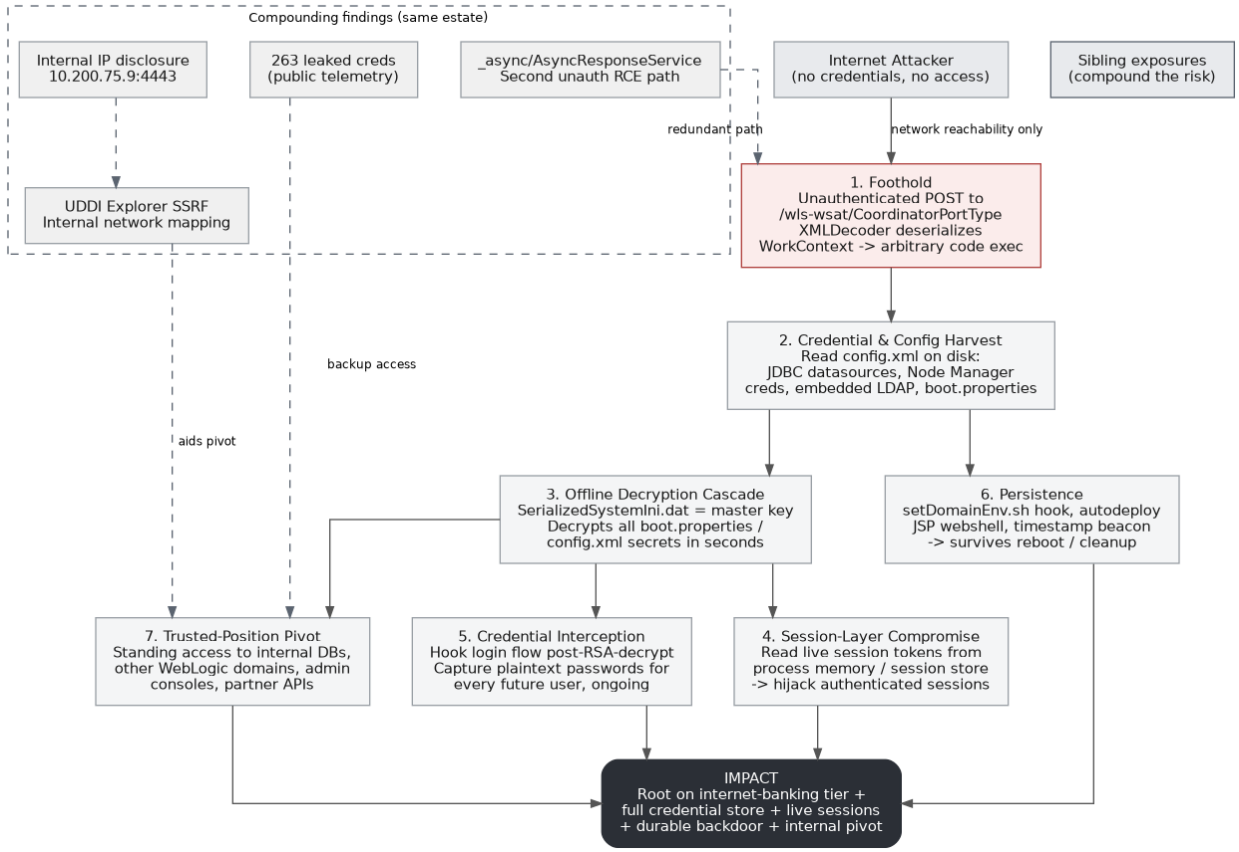


Figure 1. Full exploit chain from zero-precondition foothold to estate-wide impact, including compounding sibling findings (dashed).

The Complete Attack Chain

1. Foothold: outsider → code execution on banking tier

One unauthenticated request is enough to convert an internet-based attacker into arbitrary code execution on the internet-banking application tier — no phishing, no credential stuffing, no portal brute-forcing. In the lab replication, a single POST resulted in a file dropped on disk, process identity captured, and uid=0(root) confirmed within seconds.

2. Credential and configuration harvest

Once an attacker has code execution on a WebLogic node, the estate's central configuration becomes an immediate target. From one file — config.xml — an attacker can harvest JDBC datasource definitions (hostnames, ports, service names, usernames, and encrypted passwords for every database the node talks to — core-banking schema accounts on Finacle deployments), Node Manager credentials used to control and restart other WebLogic servers, embedded LDAP credential stores for the internal directory, and boot.properties containing AES-encrypted administrator username/password pairs for the admin console.

3. Offline decryption cascade

Those AES-encrypted blobs look safe on paper, but the domain's SerializedSystemIni.dat file is the master key for all of them, lives at a fixed path, and is trivially retrievable from the compromised host. Public tooling can take SerializedSystemIni.dat, config.xml, and boot.properties and decrypt every stored secret in seconds. In the replicated chain this produced SerializedSystemIni.dat, boot.properties, and all encrypted credential blocks in plaintext — including admin console logins that remain valid even after the original RCE endpoint is patched or WAF-blocked.

4. Session-layer compromise

This tier terminates live internet-banking sessions. With code execution here, an attacker can read active users' session tokens from process memory or session stores and hijack live sessions without ever touching passwords or OTPs. Fraud and monitoring systems focused on login anomalies never fire, because there is no anomalous login — only stolen session state.

5. Traffic interception at the decryption point

Client-side password protection (virtual-keyboard input encrypted via RSA to fldPassword) ends exactly at this server, where private-key operations happen and plaintext credentials exist in memory post-decryption. An attacker with code execution can hook the login flow to log plaintext credentials at the moment of decryption, persistently collecting every future user's credentials — and potentially reach other flows that reuse the same credentials. The node becomes a silent credential vacuum.

6. Persistence

Without deliberate hunting, one successful exploit becomes a long-term, self-healing backdoor. Validated techniques include startup script hooks (e.g. setDomainEnv.sh) that re-insert a webshell or beacon on every restart, autodeploy-directory abuse to re-deploy malicious JSPs if removed, and timestamp-based beacons confirming continued control. All of these live inside supported, administrator-touched mechanisms, so unless a bank explicitly hunts for malicious modifications to startup scripts and deployment directories, the backdoors survive reboots and routine maintenance.

7. Trusted-position pivot inward

This edge tier sits on the trusted side of the bank's perimeter, with standing connections to internal databases (via harvested JDBC datasources), other WebLogic domains and admin consoles (via Node Manager and admin credentials), and internal partner systems documented in DNS and service discovery. The UDDI SSRF endpoint compounds this further, providing open/closed/host-dead signatures across internal ranges, service fingerprinting, and a precise internal topology map — even without RCE. With credentials plus trust position, the path to internal databases, internally reachable VPN concentrators, and other tiers of the environment is straightforward.

Who Actually Exploits This Class

This is not an esoteric bug class; it is actively and systematically targeted.

Actor	Behavior
Ransomware crews / initial access brokers	Continuously scan for exposed WebLogic deserialization endpoints; sell or use access in enumerate → lateral move → mass encrypt → double-extort playbooks. Qilin's listing of Habib Bank AG Zurich (Nov 2025) shows this brand family is already in scope.
APT actors	Prefer quiet, long-dwell persistence on exposed middleware; use credential harvesting and session hijacking for espionage and sanctions-evasion financing; favor exactly this kind of blind, hard-to-detect RCE.
Financially motivated cybercrime groups	Monetize quickly via session hijack and payment tampering; favor short dwell times and low-signal exploit paths like this one.
Opportunistic botnets	Run mass scans across entire ASNs for /wls-wsat and related paths; conscript exposed endpoints into crypto-mining, DDoS, or resale inventories within days of new scans.

Why Detection Is Likely To Fail

- **Blind execution** — the vulnerable service returns HTTP 500 whether the payload succeeded or failed. Naive WAFs and response-based monitors mostly see "errors" and treat them as blocked traffic.
- **Encrypted transport** — all traffic moves over HTTPS, fronted by Imperva. Network IDS sensors outside the WAF only see ciphertext and cannot inspect headers or payloads.
- **Post-exploitation camouflage** — once inside, the attacker's activity (reading configuration files, running java commands, interacting with admin APIs) looks like normal WebLogic behavior. Without a strong baseline, anomalies do not stand out.
- **Persistence through legitimate mechanisms** — the replicated backdoor leveraged setDomainEnv.sh, a script administrators legitimately edit and tune. Few SOCs run change-detection on that file outside the most mature environments.
- **No prior baseline** — anomalous /wls-wsat usage cannot be meaningfully alerted on if its existence or reachability was unknown. That this endpoint is live at all on an internet-banking edge strongly suggests configuration drift that bypassed earlier review.

Compounding Findings In This Environment

The wls-wsat exposure is not a single, neatly isolated bug; it sits in an ecosystem of weaknesses that amplify its effect. Four sibling issues in this engagement materially compound the risk:

Sibling finding	How it compounds the risk
_async second path	A separate, unauthenticated RCE in the same family. Defenders must remediate both paths completely and correctly; attackers need only one to succeed — redundancy favors offense.
UDDI SSRF oracle	Provides internal network mapping (open ports, services, dead hosts, content types), letting any foothold move with precision instead of brute-forcing internal ranges. Remains useful even if wls-wsat is temporarily patched.
Internal IP disclosure (10.200.75.9:4443)	Offers a free, concrete starting point for SSRF scanning and confirms the presence and style of RFC1918 topology behind the edge.
263 leaked credentials (public telemetry)	A separate, independent path into the same estate via captcha-free portals — giving adversaries a race condition in their favor: whichever path (credential abuse or RCE) succeeds first becomes the primary access route, the other a backup.

Overlaying all of this is a meta-finding: Imperva's virtual patching on this estate is bypassable. The semicolon-path trick demonstrated against ToolPane proves that edge normalization has exploitable gaps — which erodes confidence in “we'll just WAF it” as a durable fix.

In short: this is a zero-precondition, blind-executing, root-yielding entry point on the money-movement tier, already chained in practice into credential store decryption, session theft, durable persistence, and internal pivoting — validated end-to-end on an identical build and currently exposed behind the WAF.